



**UNIVERSIDADE DO ESTADO DO RIO GRANDE DO NORTE - UERN
CURSO DE TECNOLOGIA EM SISTEMAS PARA INTERNET
DESENVOLVIMENTO DE SISTEMAS CORPORATIVOS**

**MARCOS PAULO PIRES SILVA
20240027679**

**ESTUDO DE CASO: DESENVOLVIMENTO DE UM SISTEMA CORPORATIVO
PARA A EMPRESA FICTÍCIA TERRA VERDE**

**MARCELINO VIEIRA
2025**

1 DEFINIÇÃO DO CENÁRIO DA EMPRESA FICTÍCIA

A Terra Verde é uma empresa fictícia do setor de agronegócio e distribuição de produtos. Atualmente, a empresa já possui um sistema para registrar seus produtos e estoque. Porém, para manter sua operação logística e comercial eficiente, a empresa necessita de um sistema simples e centralizado apenas para gerenciar de forma clara e separada seus dois tipos de parceiros comerciais: Clientes (Pessoa Física - CPF) e Fornecedores (Pessoa Jurídica - CNPJ). O sistema proposto, denominado Sistema de Gestão de Parceiros, foi desenvolvido como uma aplicação Java web local para unificar o cadastro, a consulta, atualização e exclusão de parceiros, além da geração de um relatório completo.

2 APRESENTAÇÃO DA PROPOSTA DE DESENVOLVIMENTO DO SISTEMA

A solução proposta é um Sistema de Gestão de Parceiros implementado como uma aplicação Full-Stack moderna em Java. Assim, as tecnologias definidas para o projeto foram: framework Spring Boot para construir o Backend, HTML 5, CSS 3 e JavaScript para o Frontend e, para o banco de dados, o PostgreSQL foi o escolhido. Além disso, a arquitetura adotada foi a de três camadas, o MVC. Logo, o Frontend é a Camada de Apresentação, o Backend é a Camada de Lógica de Negócio e Banco de Dados, a Camada de Persistência.

Em relação à Segurança e Acesso ao sistema, o funcionário da empresa encarregado de operar o sistema, pode cadastrar um novo administrador se necessário e fazer login com usuário e senha. Ademais, a validação das credenciais ocorre no Backend, que utiliza o Spring Security para garantir que a senha fornecida seja validada de forma segura com a criptografia BCryptPasswordEncoder.

Desse modo, o objetivo principal é oferecer uma interface intuitiva, um Dashboard, para o usuário logado, com opções de fácil acesso, permitindo o registro, a consulta, atualização e a exclusão tanto de Clientes quanto de Fornecedores. E também, a função de geração de um relatório completo sobre a atual situação dos parceiros em formato PDF, que é baixado diretamente no computador do administrador. Ou seja, o sistema busca oferecer um CRUD completo de operações. Sendo assim, para melhor auxiliar no entendimento de todas as funcionalidades requisitadas, foi criado um diagrama de casos de uso, que pode ser visualizado na Figura 1, logo abaixo.

2.1 Diagrama de Casos de Uso do Sistema

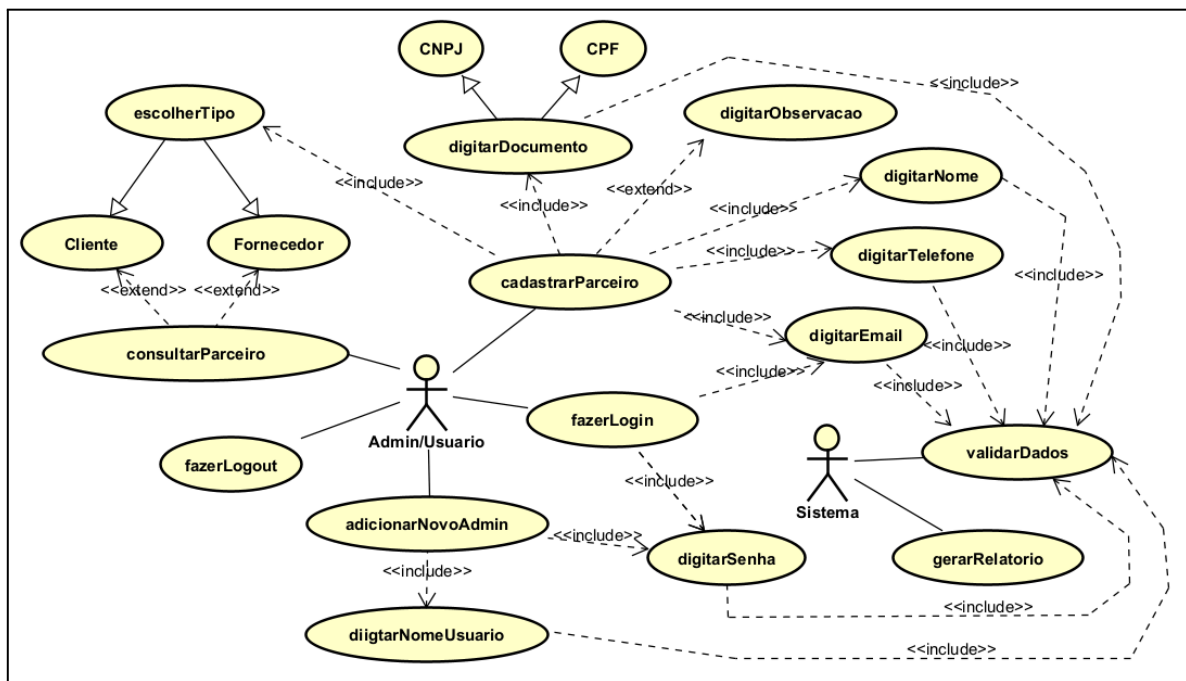


Figura 1 - Diagrama de casos de uso do sistema Gestão de Parceiros.

3 ESTRUTURA DO PROJETO

A aplicação segue o padrão de desenvolvimento Full-Stack, segregada em duas grandes áreas (Backend e Frontend) e estritamente estruturada no modelo MVC (Model-View-Controller), utilizando o Spring Boot para o lado servidor. A organização do código em pacotes lógicos (model, repository, service, controller) garante a separação de responsabilidades e a manutenção do sistema.

Essa arquitetura pode ser detalhada da seguinte forma:

1. **Modelo (Model):** Representa a camada de dados e lógica de negócio. É composta pelas classes model (Parceiro.java, Usuario.java – as entidades de dados), pelas interfaces repository (ParceiroRepository.java, UsuarioRepository.java – responsáveis pela comunicação direta com o banco de dados PostgreSQL) e pelo pacote service (ParceiroService.java – onde se encontra a lógica de negócio e regras de manipulação de dados, como a geração do relatório PDF).
2. **Controlador (Controller):** É a camada de interface que recebe as requisições HTTP do Frontend e delega a tarefa correta ao Modelo (Service/Repository). É composta pelas classes controller (ParceiroRestController.java, AuthRestController.java, RelatorioRestController.java), que expõem os endpoints da API REST para a aplicação.
3. **Visão (View):** É a camada de apresentação que interage com o usuário. Na arquitetura desacoplada do projeto, a View é composta por todos os arquivos

estáticos do pacote resources/static. Isso inclui os arquivos HTML (index.html, login.html, etc. – a estrutura da interface), os arquivos CSS (o estilo da interface) e, fundamentalmente, o JavaScript puro (app.js), que orquestra a interface, gerencia o estado de autenticação (localStorage) e faz as chamadas assíncronas (fetch) aos endpoints do Controlador.

Resumindo, o Backend (Java) concentra-se nas camadas Model e Controller, enquanto o Frontend (static) atua como a camada View inteligente. Abaixo, na Figura 2, está ilustrada a estrutura de pastas e arquivos principais do projeto, contendo a descrição de cada item.



Figura 2 - Estrutura MVC de pastas e arquivos principais do projeto com descrição.

3 TRANSAÇÕES DA EMPRESA - MODELAGEM DO SISTEMA

A modelagem de dados foi simplificada para focar na entidade central, o Parceiro, que é uma entidade polivalente.

3.1. Modelo de Dados (Backend - Java)

1. Entidade Parceiro: É a classe fundamental que armazena os dados comuns a Clientes e Fornecedores: nomeOuRazaoSocial, documento (CPF/CNPJ),

email, telefone e observacoes. O diferencial é o campo tipo, que armazena o valor "CLIENTE" ou "FORNECEDOR", permitindo que uma única tabela gerencie ambas as categorias.

2. Entidade Usuario: Utilizada para armazenar as credenciais de acesso (username e password em hash), crucial para o módulo de autenticação.
3. Repositórios (DAO): Interfaces (ParceiroRepository e UsuarioRepository) que estendem o JpaRepository, permitindo a persistência e busca de dados com métodos de alto nível, como consultas por tipo e contagens para o Dashboard.

3.2. Transações e APIs (Backend - Spring Boot)

O sistema expõe diversas APIs REST (@RestController) para o Frontend:

1. ParceiroRestController: Responsável pelo CRUD de parceiros, listagem de Clientes e Fornecedores separadamente, e pela API /relatorio que fornece as contagens de parceiros para o Dashboard.
2. AuthRestController: Responsável pelos endpoints de /login e /register, gerenciando a autenticação e a criação de novos usuários no sistema.
3. RelatorioRestController: Dedicado a tarefas específicas de relatórios, expondo o endpoint /parceiros-pdf que chama a lógica de serviço para gerar o arquivo binário.

4 CODIFICAÇÃO DAS FUNCIONALIDADES

A codificação das funcionalidades é dividida entre as duas camadas, garantindo a separação de responsabilidades:

4.1. Funcionalidades do Backend

1. Geração de Relatório PDF: A lógica complexa de manipulação de documentos foi isolada no ParceiroService. Utiliza a biblioteca iText para coletar todos os parceiros do banco de dados e formatar os dados em tabelas, gerando um arquivo PDF que é então enviado como array de bytes para o Frontend.
2. Tratamento de Erros: O ParceiroRestController inclui tratamento de exceções específicas, como DataIntegrityViolationException, para retornar mensagens claras ao usuário quando houver duplicidade de e-mail ou documento.
3. Segurança e Configuração: O SecurityConfig garante que as senhas sejam hashadas e permite que o Frontend acesse as APIs (CORS configurado), além de injetar um usuário admin/admin123 inicial para garantir o primeiro acesso.

4.2. Funcionalidades do Frontend

1. Controle de Fluxo e Autenticação: O arquivo app.js gerencia todo o fluxo de navegação. A função checkAuth() é central, pois ela:
 - a. Verifica se o usuário está logado (no localStorage);
 - b. Implementa o redirecionamento essencial que corrigiu o problema da navegação: se o usuário estiver logado e tentar acessar a raiz (/), é imediatamente redirecionado para o /index.html, garantindo que o Dashboard carregue corretamente;
 - c. Redireciona usuários não autenticados para a página de login;
2. CRUD e Dashboard: O app.js contém as funções para:
 - a. Fazer as requisições fetch para criar, editar e excluir parceiros.
 - b. Renderizar as tabelas de lista_cliente.html e lista_fornecedor.html.
 - c. Chamar a API para loadDashboardData() no index.html, preenchendo os contadores de Clientes, Fornecedores e o Total.
 - d. Geração de Relatório: A função gerarRelatorioPdf() faz uma requisição GET para a API de relatórios e trata a resposta como um Blob, forçando o download do arquivo PDF diretamente no navegador do usuário.

5. FLUXO DE NAVEGAÇÃO DO SISTEMA

Uma vez logado e na página principal (index.html), o usuário pode navegar para as áreas de gestão por meio de links diretos, realizando as principais operações CRUD (Criação, Leitura, Atualização e Exclusão):

1. Dashboard (index.html): O ponto central que exibe o resumo de Clientes e Fornecedores, obtido pelo GET /api/parceiros/relatorio.
2. Visualização (Listas): O usuário acessa as páginas lista_cliente.html ou lista_fornecedor.html, que chamam respectivamente GET /api/parceiros/clientes ou GET /api/parceiros/fornecedores.
3. Criação/Atualização (Cadastro): A página cadastro.html é usada para ambas as operações.
 - a. Criação: Envia um POST para /api/parceiros.

- b. Edição: Carrega o parceiro específico com GET `/api/parceiros/id/{id}` e salva as alterações com um PUT para `/api/parceiros/{id}`.
- 4. Exclusão: As listas permitem acionar a exclusão, que faz uma chamada DELETE para `/api/parceiros/{id}`.
- 5. Relatório PDF: O botão de relatório no Dashboard aciona a rota GET `/api/relatorios/parceiros-pdf`, que gera o documento binário e força o download no navegador.

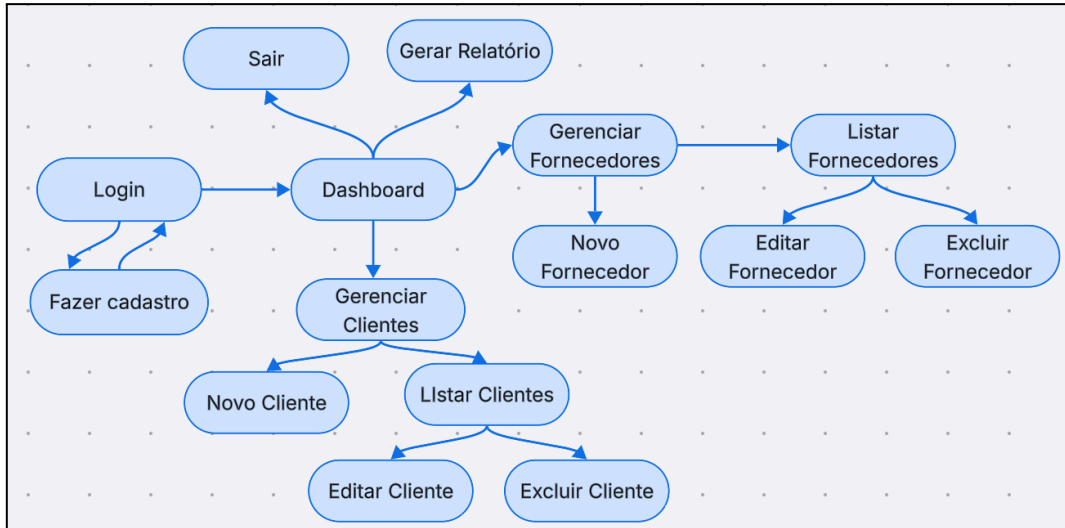


Figura 2 - Fluxo de navegação do sistema Gestão de Parceiros.